

Compilador de Intenção para o Herus — até onde vai e como chegar

Síntese a partir de três fontes desta conversa: o estado real do repositório `SummaArs/herus`, a proposta neuro-simbólica anterior (refutada), e a proposta Compilador de Intenção que você escreveu para a OonCore, com protótipo medido. 24 de agosto de 2026.

0. Por que este documento é diferente do anterior

A proposta neuro-simbólica que analisamos antes tentava resolver um problema que o Herus não tem — PLN aberto, geração livre — com peças que nem cabiam no hardware. A proposta da OonCore é de outra natureza: é sua, é medida, e inclui os pontos em que a primeira estimativa estava errada (você revisou a compressão de $\sim 10x$ para $3-7x$ medido, publicamente, sem esconder). Isso não garante que o padrão funcione no Herus — domínio e hardware são diferentes — mas muda a pergunta. Não é "isso é real?", é "o que atravessa e o que não atravessa?".

1. A tese, traduzida

Sua frase na OonCore: **"Neural compreende. Símbolo garante."** O modelo só entra onde a entrada é suja e ambígua; nunca decide se o resultado está certo — ele palpa, o símbolo aceita ou rejeita.

É a mesma tese que você já defende para o Herus: raciocínio livre resolvido por álgebra simbólica, sem LLM hospedada. A diferença é que agora você tem uma medição real de quanto essa estratégia vale, em domínio análogo: 98,0% de fidelidade contra produção, zero ausência, portão que pega 44 de 44 mutações inválidas depois de fechada uma classe de lacuna. É a primeira evidência empírica que você tem, de qualquer fonte, de que "símbolo garante" composta na prática e não só na cabeça.

2. As seis camadas, remapeadas para o Herus

Camada	Papel na OonCore	Equivalente no Herus	Custo
C1 – Inferência determinística	Tipo, obrigatoriedade e enum tirados da planilha que o cliente já tem	Contexto físico já disponível (hora, movimento, cartão anterior) usado pra pré-preencher ou restringir campos, sem interpretar nada	zero
C2 – Catálogo de padrões	7 templates de Central, preenchidos com os 3 manifestos completos	Catálogo fechado de formas de cartão ("cheguei", "atraso N min", "preciso de ajuda", "encontro em X") – hoje é conceito, não catálogo preenchido	zero
C3 – Modelo barato escreve a spec	LLM hospedado escreve ~200 linhas de spec compacta	A analogia quebra de propósito aqui – ver §3. Vira classificação VSA do gesto/fala curta contra o catálogo fechado de C2, não geração	zero (VSA)
C4 – Compilador determinístico	spec → 4 manifestos JSON	intenção reconhecida + slots → cartão de contexto tipado, schema fixo	zero
C5 – Portão	validadores do Core (biblioteca) + camada de referência cruzada	validação de schema do cartão + referência cruzada (contato/local/cartão anterior existe) + limiar de ambiguidade – já é o espírito dos seus docs 20–23	zero

Camada	Papel na OonCore	Equivalente no Herus	Custo
C6 — Corpus	pares (intenção→spec→manifesto) verificados, realimentam C2 e C3	pares (gesto/fala→cartão) confirmados pela pessoa — ver ressalva de privacidade em §3	zero

"Modelo caro" na OonCore é o Fábio pagando sessões de raciocínio de fronteira; no Herus é você. A regra que você escreveu pra ele vale igual: sessão cara nunca entrega um cartão específico, entrega catálogo novo, regra de compilador nova ou validador novo.

3. Onde a analogia quebra

- **A OonCore parte de schema e validadores já prontos e maduros.** O Herus não tem isso pronto — tem que construir, e é exatamente o que os docs 17 a 34 do seu repo já estão fazendo. A prova da OonCore valida a estratégia; não substitui o trabalho de desenhar o vocabulário do zero.
- **C3 na OonCore pressupõe API hospedada.** No Herus isso contraria o princípio central — offline, soberano, sem depender de LLM hospedada. Por isso C3 muda de natureza: classificação sobre um conjunto fechado, não geração de texto. Modelo local só entra depois de Fase 0 validada e do orçamento de memória do seu doc 40 confirmado em hardware real — nunca antes.
- **O corpus da OonCore é dado de negócio. O do Herus é dado pessoal.** Acumular pares confirmados só faz sentido dentro da política de memória seletiva já desenhada — local, com consentimento explícito, nunca saindo do dispositivo.
- **Nada disso resolve o risco físico do Herus.** Alcance, energia, ergonomia, reconhecimento de fala em hardware embarcado continuam sem medição. É contribuição de camada de software sobre uma base ainda não validada fisicamente — não substitui a Fase 0, só dá a ela um alvo mais claro do que compilar.

4. Até onde o Herus pode chegar no curto/médio prazo

Não é um Herus que conversa livremente — essa ambição já foi descartada na proposta anterior. É um Herus que reconhece um conjunto fechado, mas extensível com disciplina, de intenções físicas e vocais curtas; compila cada uma, deterministicamente, num cartão de contexto tipado; nunca aceita um cartão que não passe no portão; nunca envia ou guarda nada sem confirmação física; e consegue **provar** propriedades específicas por teste adversarial — por exemplo, "a recuperação nunca devolve ambíguo sem avisar", testado contra milhares de mutações, do mesmo jeito que o portão da OonCore prova que manifesto inválido nunca passa. Você já tem o hábito de engenharia pra isso: `prove.sh`, quarentena de boot, oráculo de recuperação já são, em espírito, o mesmo arnês adversarial que fechou a classe de lacunas na OonCore.

5. Até onde seria ousado ir

Você pediu ousadia — aqui está o teto real, não o teto morno. A ambição errada da proposta anterior era rivalizar com LLM em linguagem aberta. A ambição certa é outra, e cabe inteira dentro do que já está estabelecido:

VSA como linguagem composicional, não catálogo fixo.

Bind/unbind/bundle não servem só pra classificar contra uma lista fechada — servem pra representar estrutura. Um conjunto pequeno de papéis primitivos (quem, o quê, quando, onde, urgência, alvo) ligados a valores e agrupados compõe, combinatorialmente, um espaço muito maior de cartões possíveis do que qualquer catálogo preenchido à mão — sem parser neural, sem OWL2, sem nada que não caiba no hardware. É literatura real de VSA (Kanerva; os surveys de arquiteturas vetorial-simbólicas de Kleyko e coautores), não um número emprestado de outro domínio como na proposta anterior. É a versão correta de "linguagem completa": expressividade que cresce por composição de primitivos, não por vocabulário memorizado.

Modelo local que gera estrutura, nunca texto. Quando o Núcleo tiver orçamento de memória validado, o upgrade de C3 não precisa ficar classificador fixo pra sempre — pode virar um modelo pequeno que propõe novas ligações dentro do espaço de primitivos VSA. É geração, mas

geração tipada: o espaço de saída continua sendo a estrutura simbólica fechada, e o portão (C5) valida do mesmo jeito. Ganha expressividade sem nunca reabrir a porta pra conteúdo livre.

Prova formal de ponta a ponta, não só teste adversarial. Fuzzing adversarial dá confiança estatística. O passo ousado de verdade é model checking ou prova formal do núcleo de decisão fail-closed — poucos wearables de consumo têm qualquer verificação formal; praticamente nenhum tem o núcleo de decisão inteiro provado. Você já tem o material pra isso: C11, invariantes explícitos, quarentena de boot. Formalizar essas propriedades é diferencial técnico defensável, não promessa de marketing.

Malha de catálogo, nunca de dado pessoal. LoRa já é rádio de malha por natureza. A ideia mais ousada e ainda coerente com tudo isso: dispositivos Herus propagam **atualizações de catálogo** entre si — formas de cartão novas, regras de compilador novas — exatamente como um pacote de software se propaga, nunca cartões ou memória pessoal. Transforma o vocabulário do Herus de "o que um dispositivo aprendeu sozinho" em "o que uma rede de dispositivos comprovadamente correta aprendeu junto", sem tocar em privacidade. É semente de protocolo, não só de produto.

Onde dá pra rivalizar de verdade: não linguagem aberta, classificação em vocabulário fechado. O comparativo certo nunca foi GPT-4 — é o motor de intenção on-device que já roda em qualquer assistente de voz comercial, o classificador pequeno que reconhece comando offline. Nessa comparação, HDC/VSA tem literatura real mostrando competitividade com redes neurais pequenas em classificação com pouca energia e poucos exemplos — área ativa de pesquisa (gesto, EMG, voz), não promessa vazia. A fraqueza clássica do HDC é generalizar sobre vocabulário aberto; o Herus, por desenho, nunca precisa de vocabulário aberto. A fraqueza quase não se aplica ao seu caso.

Memória associativa VSA generaliza sem gerar. Um item memory com centenas ou milhares de exemplares (frase → forma de cartão) permite busca por vizinho mais próximo no espaço hiperdimensional. Alguém fala de um jeito que você não previu, mas perto o bastante de um exemplar já catalogado, e o sistema reconhece — sem nunca gerar uma palavra nova. É o mecanismo mais próximo de "parecer entender" que existe dentro do paradigma simbólico, e é matemática real de VSA, não força bruta.

O LLM entra em tempo de design, não em tempo real. Nada impede usar um modelo de fronteira, offline, sem pressa, pra fazer o que você teria que fazer sozinho: varrer cenários reais de uso off-grid — trilha, resgate, equipe de campo, blecaute — e propor um catálogo de formas de cartão bem mais largo do que dá pra escrever à mão. Não é "o Herus roda um LLM"; é "um LLM ajudou a projetar o Herus". A cobertura final pode rivalizar com o que alguém sentiria digitando qualquer coisa num chat — porque os casos já foram pensados, uma vez, offline, revisados por você, não a cada mensagem.

Por que "zero aprendizado" não é a aposta certa. Isso é pesquisado a sério, não é ideia nova: a Symbolica AI — laboratório de ~US\$33M, coautoria com o Google DeepMind — tenta exatamente raciocínio geral via álgebra pura (teoria das categorias, mais "simbólica" que VSA) desde 2022. A posição pública deles hoje: síntese de programa de propósito geral "permanece um dos problemas de pesquisa mais difíceis em IA", não resolvido. E mesmo eles não buscam eliminar o aprendizado — buscam unir estrutura simbólica com otimização neural, não substituir uma pela outra. Isso se repete em todo canto: até os melhores sistemas de 2026 pra raciocínio silogístico (a tarefa mais "álgebra pura" que existe) ainda usam uma LLM pra traduzir linguagem natural em lógica formal antes do solver simbólico entrar. Nenhum sistema sério e atual demonstra raciocínio livre generativo com zero componente aprendido. O "primeiro" real pro Herus não é essa versão — é aprendizado local mínimo (ajustar o embedding hiperdimensional a partir do uso do próprio dispositivo, nunca LLM hospedada) aplicado a coordenação off-grid embarcada, com prova formal do núcleo. Isso ninguém fez ainda, e cabe no ESP32-S3.

O estado da arte certo pro Herus não é "o mais inteligente". É "o único que prova que nunca falha silenciosamente". Nenhum comunicador off-grid do mercado hoje prova que nunca vaza ambiguidade sem avisar, nunca envia sem confirmação física, nunca aceita um cartão malformado. Essa é a reivindicação de estado da arte que dá pra sustentar de verdade — e, ao contrário da proposta neuro-simbólica anterior, cabe inteira no ESP32-S3 e no Núcleo, porque todo o peso está em prova e composição, não em parsers pesados.

6. Como chegar

Fase 0 (já planejada, sem mudança): dois devkits, bancada, RF, energia, critérios de corte definidos antes de qualquer coisa abaixo. Nada em software substitui isso.

Em paralelo, no host, sem depender de hardware — mesmo estilo do seu `prove.sh` atual:

- **S1 — Encher o catálogo (C2).** 5 a 8 formas reais de cartão, schema completo cada uma, nenhuma linha de VSA ainda. Mesma lição da OonCore: os templates vazios eram o item de maior retorno por hora, e não usava IA nenhuma.
- **S2 — Compilador determinístico (C4).** Intenção reconhecida + slots → cartão. Teste com comparação canonizada (ordem não é conteúdo, meça separado) — foi o que salvou a medição da OonCore de um bug de indexação.
- **S3 — Portão (C5).** Validação de schema + referência cruzada + limiar de ambiguidade, com arnês adversarial e **controle**: cartões válidos precisam continuar passando, senão o número não vale nada.
- **S4 — Classificação VSA (C3).** Bind/unbind/bundle sobre o catálogo fechado de C2. Só depois de C2 e C4 testados — sem cartão-alvo bem definido não tem o que classificar.
- **S5 — Corpus e telemetria (C6), local e consentida.** Registre cada cartão confirmado; acompanhe uma métrica só, por exemplo taxa de reconhecimento sem escalar pra pergunta de esclarecimento — seu equivalente ao "custo por Central" da OonCore, igualmente falsificável.
- **S6 — Modelo local no Núcleo (upgrade de C3), condicionado.** Só depois de Fase 0 validada em hardware e do orçamento do doc 40 confirmado. Antecipar aqui compra a mesma dívida que fine-tune antecipado compraria na OonCore.
- **S7 — Composicionalidade VSA e prova formal, horizonte longo.** Não entra na sequência acima; é o que transforma catálogo fixo em linguagem composicional e adiciona verificação formal ao núcleo fail-closed (§5). Só faz sentido depois de S1–S5 medidos e Fase 0 validada — é upgrade de teto, não de piso.

Princípio herdado direto: **código ou exceção ad-hoc pra um caso que não encaixa em nenhuma forma do catálogo é sinal de lacuna no catálogo**, não uma exceção pra programar por fora. Registre e feche a classe — foi assim

que o `crossref.js` da OonCore levou o portão de 86% pra 100% com 96 linhas.

7. Princípio operacional

Toda sessão sua no Herus devia sair com uma destas coisas: uma forma nova no catálogo, uma regra nova no compilador, um validador novo, ou uma classe de ambiguidade fechada — nunca só "resolvi esse caso específico". É a mesma regra que você escreveu pro Fábio, e importa mais ainda pra você: seu tempo está dividido entre várias frentes ao mesmo tempo, então cada sessão precisa render capacidade permanente, não trabalho braçal repetido.